

2店舗目セットアップ手順

同じアプリのコードを使って、もう1店舗ぶんの勤怠アプリを立ち上げるための手順です。

構成のイメージ

店舗A（既存）	https://kinntai-app.onrender.com	→	Supabase DB（A）	→	スプレッドシート（A）
店舗B（新規）	https://kinntai-app-b.onrender.com	→	Supabase DB（B）	→	スプレッドシート（B）
↑ 同じ GitHub リポジトリ・同じコードを共有					

- **コードは1つ**。店舗ごとの違いは Render の環境変数だけで切り替わります。
- **データベースは店舗ごとに別**。キャストも打刻もシフトも完全に分かります。
- 店舗Aの設定・データには一切影響しません。

手順1. 店舗B用のデータベースを作る（Supabase）

1. Supabase にログインし、**New project** で新しいプロジェクトを作る

- 名前は `kintai-store-b` など分かりやすいもの
- Database Password は控えておく
- Region は `Northeast Asia (Tokyo)` を選ぶ

2. プロジェクトができたなら **Project Settings** → **Database** → **Connection string** を開く

3. **URI** の接続文字列をコピーする（`postgresql://postgres:...` で始まる文字列）

- `[YOUR-PASSWORD]` の部分は、1で決めたパスワードに置き換える

※ 店舗Aと同じデータベースを使わないでください。

同じにすると両店舗のキャスト・打刻が混ざります。

テーブルは、アプリを最初に開いたときに自動で作られます。手動でのSQL実行は不要です。

手順2. 店舗B用のサービスを作る（Render）

1. Render のダッシュボードで **New** → **Web Service**

2. 既存の勤怠アプリと **同じ GitHub リポジトリ** を選ぶ

3. 以下を設定する

項目	値
Name	<code>kintai-app-b</code> （好きな名前でOK。URLになります）
Runtime	Python
Build Command	<code>pip install -r requirements.txt</code>
Start Command	<code>gunicorn app:app</code>

4. **Environment（環境変数）** に次を登録する

変数名	設定する値	必須
<code>DATABASE_URL</code>	手順1でコピーした接続文字列	●

変数名	設定する値	必須
STORE_NAME	店舗名（画面に表示されます）	●
SITE_ACCESS_CODE	店舗B用のアクセスコード（店舗Aとは別の値に）	●
ADMIN_PASSWORD	店舗B用の管理者パスワード（店舗Aとは別の値に）	●
INITIAL_CAST_MEMBERS	空のまま（キャストは管理画面から登録）	
SECRET_KEY	ランダムな文字列（Render の Generate でOK）	●
FLASK_ENV	production	●
SHEETS_WEBHOOK_URL	手順3で取得する（後から追加でOK）	
SHEETS_WEBHOOK_SECRET	手順3で決める合言葉（後から追加でOK）	
CRON_SECRET	店舗Aと同じ値（当欠自動判定用）	
GEMINI_API_KEY	店舗Aと同じキーで問題なし（シフト表のAI読み取り用）	

5. Create Web Service でデプロイする

デプロイが終わったら、発行された URL（例: `https://kintai-app-b.onrender.com`）を控えます。

手順3. 店舗B用のスプレッドシート連携

店舗Aとは別のスプレッドシートに書き込ませます。

- Google ドライブで**新しいスプレッドシート**を作る（名前: 勤怠管理（店舗B） など）
- 上部メニュー **拡張機能** → **Apps Script** を開く
- リポジトリの `apps_script.gs` の中身を**すべてコピーして貼り付け**、保存する
- 合言葉を使う場合は、先頭付近の `var SECRET = "";` にパスワード文字列を入れる
 - ここに入れた値を、Render の `SHEETS_WEBHOOK_SECRET` にも同じように設定する
- デプロイ** → **新しいデプロイ** を選ぶ
 - 種類: **ウェブアプリ**
 - 次のユーザーとして実行: **自分**
 - アクセスできるユーザー: **全員**
- 発行された**ウェブアプリURL**（`/exec` で終わるもの）をコピーする
- Render の店舗Bサービスの環境変数 `SHEETS_WEBHOOK_URL` に貼り付けて保存（自動で再デプロイされます）

動作確認: ブラウザでウェブアプリURLを開いて `kintai sheets webhook v7-summary-history` と表示されればOKです。

手順4. 自動処理を2店舗ぶんに広げる（GitHub Actions）

当欠の自動判定（毎日9時）とスリープ防止のPingを、店舗Bにも実行させます。

1. GitHub のリポジトリで **Settings** → **Secrets and variables** → **Actions** を開く
2. **Variables** タブ → **New repository variable**

- Name: STORE_BASE_URLS
- Value: 両店舗のURLをスペース区切りで並べる

```
https://kinntai-app.onrender.com https://kintai-app-b.onrender.com
```

3. **Secrets** タブの CRON_SECRET は既存のものをそのまま使う

（店舗Bの Render 環境変数 CRON_SECRET にも同じ値が入っていることを確認）

設定後、**Actions** → **Daily absence check** → **Run workflow** で手動実行し、両店舗とも HTTP 200 が返ることを確認してください。

STORE_BASE_URLS を設定しない間は、これまでどおり店舗Aだけが対象になります。

手順5. 店舗Bの初期設定と動作確認

1. 店舗BのURLを開く → アクセスコード入力画面が出る
2. 手順2で設定した SITE_ACCESS_CODE を入力
3. ログイン画面に**店舗名**が表示されていることを確認（キャスト一覧はまだ空です）
4. 「管理者ログイン」を開き、ADMIN_PASSWORD で管理者としてログイン
5. **キャスト管理**からキャストを1人ずつ登録する（送迎料金もここで設定）
6. **シフト管理**でシフトを登録（シフト表の画像・PDFからのAI読み取りも使えます）
7. キャストとしてログインし、出勤 → 退勤の打刻を試す
8. スプレッドシート（店舗B）に「全員月間集計」タブが自動作成されることを確認

環境変数まとめ

変数名	店舗ごとに変える	説明
DATABASE_URL	● 必ず別	Supabase の接続文字列。 共有すると データが混ざります
STORE_NAME	●	画面に表示する店舗名。未設定なら 「勤怠管理 / タイムカード」と表示
SITE_ACCESS_CODE	● 推奨	サイト全体のアクセスコード
ADMIN_PASSWORD	● 推奨	管理者ログインのパスワード
SHEETS_WEBHOOK_URL	● 必ず別	書き込み先スプレッドシートの Apps Script URL
SHEETS_WEBHOOK_SECRET	任意	Apps Script 側の SECRET と揃える合言葉

変数名	店舗ごとに变える	説明
SECRET_KEY	●	セッション暗号化キー。サービスごとに自動生成でOK
INITIAL_CAST_MEMBERS	●	初回起動時に自動登録するキャスト名（カンマ区切り）。空なら登録なし
CRON_SECRET	同じでOK	当欠自動判定のトリガー用トークン
GEMINI_API_KEY	同じでOK	シフト表AI読み取り用のAPIキー
FLASK_ENV	同じ	production

注意点

- **INITIAL_CAST_MEMBERS** が効くのは、そのDBが空のときの初回起動時だけです。
一度キャストが登録された後に値を変えても反映されません。以降は管理画面のキャスト管理から追加・削除してください。
 - 管理者アカウント（ユーザー名 `admin`）は初回起動時に自動作成されます。
 - 3店舗目以降も同じ手順です。Render のサービスと Supabase のプロジェクトを増やし、`STORE_BASE_URLS` にURLを追加するだけで対応できます。
 - 営業日の定義（20:00～翌9:00）、遅刻判定、勤務時間の15分刻み計算などの業務ロジックは全店舗共通です。
- 店舗ごとに変えたい場合は別途対応が必要です。